Idiomatic Modern C++ for Linux

Week 5: References and Pointers

Today’s agenda

- Intro to lvalues and rvalues
- references
- pointers
- passing by reference
- passing by address
- returning by reference / address
- in and out parameters
- std::optional

• lvalues and rvalues

- An lvalue is an expression that evaluates to an ***identifiable object***
- An rvalue is any expression that is ***not an lvalue.***
- An rvalue evaluates to ***a value***
- Const and constexpr variables signify **non-modifiable** lvalues

```
// 5 is an rvalue expression
int x{ 5 };

// 1.2 is an rvalue expression
const double d{ 1.2 };

// x is a modifiable lvalue expression
int y { x };

// d is a non-modifiable lvalue expression
const double e { d };

// return_an_int() is an rvalue expression
// (since the result is returned by value)
int z { return_an_int() };

// x + 1 is an rvalue expression
int w { x + 1 };

// the result of static casting d to an int
// is an rvalue expression
int q { static_cast<int>(d) };
```

• lvalues and rvalues

- Lvalues will implicitly convert to rvalues in places where an rvalue is expected (the lvalue is evaluated to produce its value)
- rvalues cannot be converted to lvalues
- e.g. operator= expects its left operand to be a modifiable lvalue expression

```
// y is evaluated, "converting" the lvalue to an rvalue
x = y;

// 9 and 10 are rvalues, operator+ returns an rvalue
cout << 9 + 10;

// '52' is an rvalue, not a modifiable lvalue expression
// and assignment requires the left operand to be a modifiable lvalue
52 = x;
```

• lvalues and rvalues

- Rule of thumb when figuring out if an expression is an lvalue or an rvalue:
  - Does it persist beyond the expression?
  - Does it evaluate to a function, identifiable object, or expression?
- If so, it’s probably an lvalue!

• Types of references

- Modern C++ contains two types of references:
  - lvalue references
  - rvalue references
- rvalue references will be covered in a future lesson on move semantics

Lvalue References

- reference – alias to an existing object
- Any operation on the reference is applied to the object being referenced
- Used almost everywhere in the language

```cpp
// normal int
int a {4};

// non-const reference to int.
int& aref(a);

// should print "4, 4"
cout << a << ", " << aref << endl;

a = 3;
// should print "3, 3"
cout << a << ", " << aref << endl;

// operations on the reference apply to the object being referenced.
aref = 126;

// should print "126, 126"
cout << a << ", " << aref << endl;
```

More about lvalue references

- References must always be initialized and bound to a referent
- Non-const objects can be referred to by an ***lvalue reference to a const int***
- The referent in non-const references must be mutable. Otherwise, we would be able to modify const variables with non-const references!

```
// invalid: references must always be initialized
int& uninitialized_ref;

// valid: lvalue reference to a const int.
const int& caref {a};

// invalid: can't modify a reference to const
//caref = 12;

const int MY_CONSTANT {42};

// invalid: non-const references can only be bound to mutable values
int& non_const_ref{MY_CONSTANT};
```

More about lvalue references

- References are typically only bound to values of the same type
- References can’t be “reseated” to refer to another object
- References and referents have independent lifetimes. One can be destroyed before the other
- Will result in a ***dangling reference*** if the underlying object dies before the reference. This constitutes undefined behavior

```
int xx {42};

// invalid: can't convert from int to double&
double& xref {xx};

int x { 5 };
int y { 6 };

// ref is now an alias for x
int& ref { x };

// assigns 6 (the value of y) to x (the object being referenced by ref)
// This does NOT change ref into a reference to variable y!
ref = y;
cout << x << endl; // user is expecting this to print 5

int q {5};
{
    int& r {q};
} // r dies here, q is unaware
```

Passing by reference

- When writing args for function signatures, by default objects are passed **by value**, ***which implies a copy***
- Some objects are expensive to copy
- We can avoid the copy by passing by reference or const reference
- Unless you need to modify the object in the function itself, always pass by const reference

```cpp
// calling on a vector results in a copy
void print_value(std::vector<int> vec) {
    for (const auto& e : vec) {
        cout << e << endl;
    }
}    // vec is destroyed here
```

```cpp
// no copy created, underlying object is referenced
// (can't be called on const refs)
void print_ref(std::vector<int>& vec) {
    for (const auto& e : vec) {
        cout << e << endl;
    }
    // Warning! by passing as a non-const ref,
    // the underlying object can be mutated
    // if it wasn't declared const
    vec.emplace_back(35);
}
```

```cpp
// no copy created, underlying object is referenced
// and immutable (since we pass as a const reference)
void print_const_ref(const std::vector<int>& vec) {
    for (const auto& e : vec) {
        cout << e << endl;
    }
    vec.emplace_back(35); // invalid, vec is a const ref
}
```

Pointers

- Pointers are a compound type consisting of a ***memory address*** and the ***underlying data it points to***
- Pointers store memory addresses. We can get the address of an object with **operator& (returns a pointer)**
- We can ***dereference*** the pointer (access its underlying data) with ***operator****
- The size of a pointer depends on the size of memory addresses for a given architecture (e.g. 32 bits on a 32 bit machine, 64 bits on a 64 bit machine)

```cpp
int my_int {31};

// we initialize ptr_to_int by getting the address of my_int
// (with operator&)
int* ptr_to_int = &my_int;

cout << my_int <<" lives at address "<< std::hex << ptr_to_int << endl;
```

Null pointers

- Pointers can be either uninitialized, or initialized as nullptr or with an address
- Dereferencing uninitialized or null pointers leads to undefined behavior
- Always initialize your pointers or make sure the pointer you’re dereferencing leads to valid data

```
int x{5};

// an uninitialized pointer (holds a garbage address)
int* ptr;
// a null pointer
int* ptr2{};
// a pointer initialized with the address of variable x
int* ptr3{&x};
```

Checking for null pointers

- Old C style syntax allows for the pointer literals 0 and NULL
- 0 can be interpreted as an integer literal or null pointer literal
- NULL can be interpreted as 0, 0L, ((void*)0), or something else entirely
- Nowadays, the nullptr keyword should be preferred, or you can check the truthiness of a pointer
- evaluates to false if the pointer is null

```cpp
// Outdated pointer literals
if (ptr2 == NULL) {
    cout << "ptr2 is null" << endl;
}
if (ptr2 == 0) {
    cout << "ptr2 is null" << endl;
}
```

```cpp
// Modern C++ way of checking pointer validity
if (!ptr2) {
    cout << "ptr2 is null" << endl;
}
if (ptr2 == nullptr) {
    cout << "ptr2 is null" << endl;
}
```

Pointers and const

- A non-const pointer (e.g. int* ptr) can be assigned another address to change what it is pointing at.
- A const pointer (e.g. int* const ptr) always points to the same address, and this address can not be changed.
- A pointer to a non-const value (e.g. int* ptr) can change the value it is pointing to. These can not point to a const value.
- A pointer to a const value (e.g. const int* ptr) treats the value as const when accessed
  - Can’t change the value it is pointing to
  - These can be pointed to const or non-const l-values (but not r-values, which don’t have an address).

```
int val {99999};
int val2 {702425};
// pointer to a const int.
// The pointer can point elsewhere,
// but the data it points to can't be modified
const int* ptr2const = &val;
ptr2const = &val2;
// *ptr2const = 4;

// const pointer to an int.
// the pointer can't point elsewhere,
// but the data it points to can be modified
int* const constptr = &val;
// constptr = &val2;
*constptr = 5;
```

Passing by address

- Works very similarly to passing by reference, except pointers are used instead of references
- Care should be taken to ensure the pointer passed in is not null before operating on it
- Most of the time, you should prefer pass by const reference instead of pass by address
- Can be used to signify an optional value (but we'll talk later about a better way)

```cpp
// calling on a vector results in a copy
void print_value(std::vector<int> vec) {
    for (const auto& e : vec) {
        cout << e << endl;
    }
}    // vec is destroyed here

// no copy created, underlying object is referenced
// (can't be called on const refs)
void print_by_addr(std::vector<int>* vec) {

    if (!vec) {
        return;
    }

    for (const auto& e : *vec) {
        cout << e << endl;
    }
    // Warning! by passing as a non-const address,
    // the underlying object can be mutated
    // if it wasn't declared const
    vec->emplace_back(35);
}

// no copy created, underlying object is referenced
// and immutable (since we pass as a const ptr)
void print_by_const_addr(const std::vector<int>* vec) {

    if (!vec) {
        return;
    }

    for (const auto& e : *vec) {
        cout << e << endl;
    }
    //vec.emplace_back(35); // invalid, vec is a const ptr
}
```

Returning by reference / address

- Functions can also use references and addresses as return values
- Important! The object that the ref / addr points to must exist beyond the scope of the function
- Non-const static local variables returned by reference should be avoided most of the time (except when you want to do a singleton pattern)

```cpp
// we can return by const or non-const reference,
// but the object it refers to must exist after the function returns.
const std::string& get_program_name() {
    // only initialized once, then reused on consecutive function calls
    static const std::string program_name{"references"};
    return program_name;
}

// In general, non-const static local variables returned by reference
// are not idiomatic and should be avoided.
int& get_next_id() {
    static int id_x {0};
    ++id_x;
    return id_x;
}

// It's generally okay to return reference parameters by reference
const std::string& first_alphabetical(const std::string& a, const std::string& b) {
    // We can use operator< on std::string to determine which comes first alphabetically
    return (a < b) ? a : b;
}
```

in and out parameters

- We can pass objects by address or reference, and mutate non-const objects inside of functions
- If you have to use an “out parameter”, it’s best to prefer a non const reference if it’s non-optional

```
bool some_operation_that_can_fail(const MyInputData& in, MyOutputData& out) {

    if (could_fail(in)) {
        out = MyOutputData();
        return true;
    }

    return false;
}
```

std::optional

- As of C++17, there’s a handy way to signify something that may or may not possess a value
- Preferable to out parameters
- Intent of the return type made clear
- When used as an arg, it makes a copy of its argument. Use for args that would have been passed by value anyway.
- Can mimic a reference via std::reference_wrapper
- If T is expensive to copy, use const T*

```cpp
std::optional<MyOutputData> some_operation_that_can_fail(const MyInputData& in) {

    if (could_fail(in)) {
        return MyOutputData();
    }

    return std::nullopt;
}
template<typename T>
void print_something_that_might_not_exist(std::optional<const T> obj = std::nullopt) {
    if (obj.has_value()) {
        cout << *obj << endl;
    } else {
        cout <<"obj is null" << endl;
    }
}
```

Additional Resources

- https://en.cppreference.com/w/cpp/language/value_category.html
- https://www.learncpp.com/cpp-tutorial/stdoptional/
- https://www.learncpp.com/cpp-tutorial/type-deduction-with-pointers-references-and-const/
- https://www.learncpp.com/cpp-tutorial/pointers-and-const/